

Multi-Tenant CalculateNDI API — Low-Level Design

Oct 2, 2026 · @Anand

Overview

Each tenant gets its own endpoint, `POST /v1/tenants/{tenantId}/calculate-ndi`, and its own calculator class behind one shared interface. Shared code resolves the tenant, authorizes the caller and dispatches; it contains no tenant-specific logic and no `if (tenant == ...)` branch. Each tenant's logic lives in its own build module, which cannot import another tenant's code.

Assumptions this design rests on:

- **Stack:** Java 21 and Spring Boot 3 for the code sketches. The pattern carries to any language with interfaces and modules.
- **Tenants are code:** each tenant's calculator ships with the service and is known at deploy time.
- **Isolation is logical:** one deployment and one database, separated by code boundaries, tenant-scoped data access and per-tenant runtime limits.
- **Calculations are synchronous:** short and CPU-bound, so there is no job or polling API.
- **Example tenants:** `uscard` and `autofinance` stand in for the real tenants.
- **NDI:** read here as net disposable income. Both NDI formulas are placeholders for the real rules.

Isolation model

Isolation is enforced at seven layers, so a mistake at one layer is caught by the next.

Layer	Shared or per-tenant	How isolation is enforced
Routing	Shared route template, one URL per tenant	Tenant id is read from the path once, by the interceptor only
Identity	Shared	Token's <code>tenant_id</code> claim must equal the path tenant, else 403

Layer	Shared or per-tenant	How isolation is enforced
NDI calculation logic	Per-tenant module	Each module implements <code>TenantCalculator</code> ; modules cannot import each other
Input and output schema	Per-tenant	Each module owns its typed input and output records
Configuration	Per-tenant	Platform hands a calculator only its own <code>tenants.<id>.*</code> settings
Data	Shared tables	<code>tenant_id</code> on every row, enforced by Postgres row-level security
Runtime capacity	Per-tenant	Separate rate limiter and bulkhead per tenant

Six rules follow from this:

1. **Resolve once.** Tenant identity is established at the edge and is immutable for the request.
2. **Look up, never branch.** Shared code finds a calculator by tenant id. It never contains `if`, `switch` or config flags keyed on a specific tenant.
3. **Depend on the contract only.** A tenant module depends on `calc-spi` and nothing else in the codebase.
4. **Keep calculators pure.** A calculator is stateless: input and settings in, result out. No database, HTTP or static mutable state.
5. **Scope every data access.** Every query, cache key and idempotency key carries the tenant id.
6. **Contain failure.** One tenant's load, slowness or bug cannot use another tenant's capacity.

API contract

Every tenant has the same two endpoints under its own path. The envelope is shared; the `input` and `result` bodies are defined by the tenant's module.

Method	Path	Purpose
POST	<code>/v1/tenants/{tenantId}/calculate-ndi</code>	Run the tenant's NDI calculation and store the result
GET	<code>/v1/tenants/{tenantId}/calculate-ndi/{calculationId}</code>	Fetch a stored result

`tenantId` is a lowercase slug matching `[a-z][a-z0-9-]{1,30}`.

Headers

Header	Required	Meaning
<code>Authorization: Bearer <JWT></code>	Yes	Token must carry a <code>tenant_id</code> claim equal to the path tenant
<code>Idempotency-Key</code>	No	Retries with the same key and body return the stored result
<code>X-Request-Id</code>	No	Echoed in the response and logs; generated if absent

Request and response

The same call for two tenants. Each tenant has its own input shape and its own formula.

```
POST /v1/tenants/uscard/calculate-ndi
{ "input": { "monthlyNetIncome": "5200.00", "monthlyHousingCost": "1500.00",
"monthlyDebtPayments": "650.00" } }
```

```
200 OK
{
  "calculationId": "7b0e4c1a-5d2f-4e7a-9a53-2f6f1c0d8e11",
  "tenantId": "uscard",
  "calculatorVersion": "uscard-1",
  "result": { "ndi": "1850.00" },
  "computedAt": "2026-10-02T15:47:00Z"
}
```

```
POST /v1/tenants/autofinance/calculate-ndi
{ "input": { "annualGrossIncome": "84000.00", "monthlyObligations":
"2100.00", "dependents": 2 } }

200 OK
{
  "calculationId": "c2a6f0de-91b4-4b0c-8a3e-0f4f7d5b6a22",
  "tenantId": "autofinance",
  "calculatorVersion": "autofinance-1",
  "result": { "ndi": "2550.00" },
  "computedAt": "2026-10-02T15:47:01Z"
}
```

Money is sent as a decimal string, never a JSON number, so no precision is lost in transit.

Errors

Errors use `application/problem+json` (RFC 9457) with a stable `type` per row.

Status	Type	When
400	<code>invalid-input</code>	<code>input</code> does not match the tenant's schema
401	<code>unauthenticated</code>	Missing or invalid token
403	<code>tenant-mismatch</code>	Token's tenant differs from the path tenant
404	<code>tenant-not-found</code>	Caller's own tenant has no calculator registered
404	<code>calculation-not-found</code>	No such calculation for this tenant
409	<code>idempotency-conflict</code>	Same <code>Idempotency-Key</code> , different body
422	<code>calculation-rejected</code>	Input is well-formed but a tenant rule rejects it
429	<code>rate-limited</code>	Tenant's quota is used up; <code>Retry-After</code> is set
503	<code>tenant-busy</code>	Tenant's concurrency limit is reached

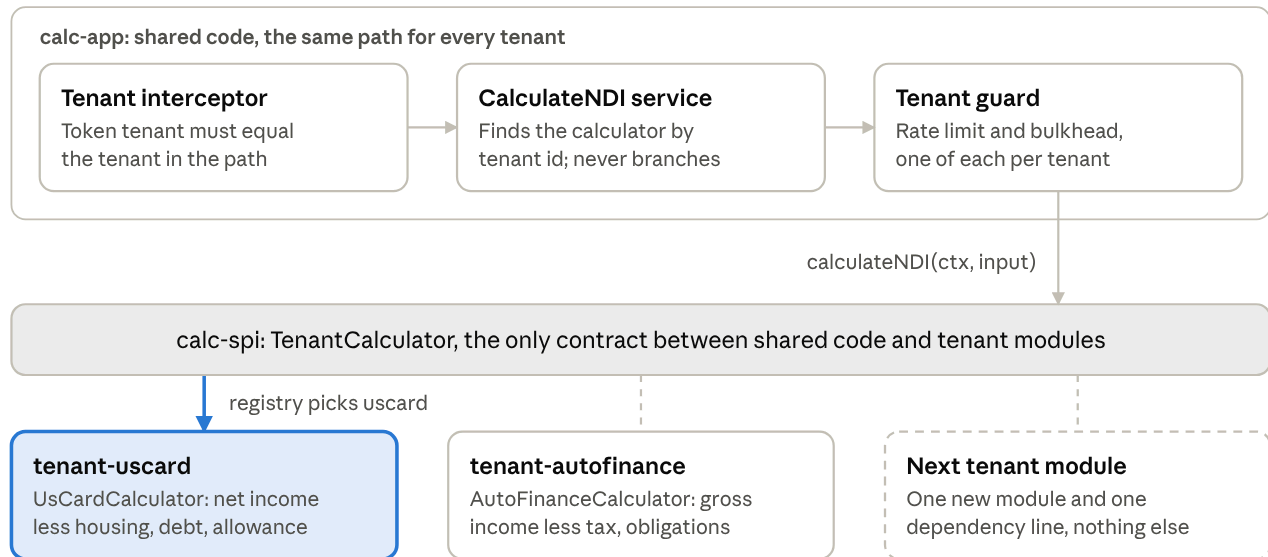
A caller with a valid token for one tenant always gets 403 on another tenant's path, whether or not that tenant exists. This keeps tenant names from being discoverable.

Request flow

A request passes through the same three shared steps for every tenant, then crosses one interface into that tenant's module.

Shared code routes the request; one tenant module calculates the NDI

Example: POST /v1/tenants/uscard/calculate-ndi. The highlighted module is the only tenant code that runs.



Tenant modules have no arrows between them: one importing another does not compile.

request flow · 3 shared steps, 1 contract, 1 module per tenant

1. The security filter validates the JWT; an invalid token gets 401 before any tenant logic runs.
2. The interceptor compares the path tenant with the token's `tenant_id`, then builds the `TenantContext` with that tenant's settings.
3. The service asks the registry for the tenant's calculator and converts `input` to the calculator's own input type.
4. The guard applies the tenant's rate limit and bulkhead, then calls `calculateNDI`.
5. The repository stores the result in a transaction scoped to the tenant, and the response is returned.

Code structure

The build has one contract module, one shared application module and one module per tenant. The compiler enforces the boundary, because a tenant module's classpath holds only `calc-spi`.

```

calc-platform/
├─ calc-spi/                com.capitalone.calc.spi                the
contract, no framework
├─ calc-app/                com.capitalone.calc.app                Spring
Boot service, shared code
├─ tenants/
│   ├─ tenant-uscard/       com.capitalone.calc.tenant.uscard
│   └─ tenant-autofinance/  com.capitalone.calc.tenant.autofinance

```

Module	Contains	May depend on	Must not depend on
calc-spi	TenantCalculator , TenantContext , TenantId , TenantSettings	JDK only	Anything else
tenant- <id>	One calculator, its input and output records, its tests	calc-spi , JDK	calc-app , other tenant modules, Spring, JDBC, HTTP clients
calc-app	Controller, interceptor, registry, service, guard, repository	calc-spi at compile scope; tenant modules at runtime scope only	Tenant classes in source

Runtime scope is the key detail. `calc-app` ships with every tenant module on its classpath, but its source cannot name `UsCardCalculator` , so a tenant-specific branch in shared code does not compile. Calculators are found at startup through Java's `ServiceLoader` .

Each tenant module can have its own code owners, so a change to one tenant's logic is reviewed by that tenant's team and touches no shared file.

Core types

One interface is the whole contract between shared code and a tenant. The contract, both calculators, the registry and the service below were compiled and run; the web-layer classes are sketches.

The contract (`calc-spi`)

```

public record TenantId(String value) {
    private static final Pattern SLUG = Pattern.compile("[a-z][a-z0-9-]{1,30}");

    public TenantId {
        if (value == null || !SLUG.matcher(value).matches()) {
            throw new IllegalArgumentException("Invalid tenant id");
        }
    }
}

/** Read-only settings for one tenant. The platform fills it from tenants.
<id>.* only. */
public record TenantSettings(Map<String, String> values) {
    public TenantSettings {
        values = Map.copyOf(values);
    }

    public BigDecimal decimal(String key, String fallback) {
        return new BigDecimal(values.getDefault(key, fallback));
    }
}

/** Who the request is for. Built once at the edge, immutable, passed
explicitly. */
public record TenantContext(TenantId tenantId, String requestId,
TenantSettings settings) {}

/** Implementations must be stateless and must not do I/O. */
public interface TenantCalculator<I, O> {
    TenantId tenantId();

    /** Recorded with every result, so a stored result can be traced to the
logic that made it. */
    String version();

    /** The tenant's own input type. The platform deserializes the request's
"input" into it. */

```

```

    Class<I> inputType();

    O calculateNDI(TenantContext ctx, I input);
}

```

`TenantContext` is passed as a parameter, not held in a `ThreadLocal`. A method that needs the tenant must ask for it in its signature, so it cannot be forgotten or leak across pooled threads.

A tenant module (`tenant-uscard`)

US Card takes monthly net income and subtracts housing, debt payments and a living allowance. Its input and output records are package-private, so nothing outside the module can use them.

```

record UsCardInput(BigDecimal monthlyNetIncome, BigDecimal
monthlyHousingCost, BigDecimal monthlyDebtPayments) {}
record UsCardResult(BigDecimal ndi) {}

public final class UsCardCalculator implements TenantCalculator<UsCardInput,
UsCardResult> {
    private static final TenantId ID = new TenantId("uscard");

    @Override public TenantId tenantId() { return ID; }
    @Override public String version() { return "uscard-1"; }
    @Override public Class<UsCardInput> inputType() { return
UsCardInput.class; }

    @Override
    public UsCardResult calculateNDI(TenantContext ctx, UsCardInput input) {
        if (input.monthlyNetIncome().signum() <= 0) {
            throw new CalculateNDIRejectedException("monthlyNetIncome must be
positive");
        }
        BigDecimal livingAllowance = ctx.settings().decimal("living-
allowance", "1200.00");

        BigDecimal ndi = input.monthlyNetIncome()
            .subtract(input.monthlyHousingCost())
            .subtract(input.monthlyDebtPayments())
            .subtract(livingAllowance)
    }
}

```

```

        .setScale(2, RoundingMode.HALF_EVEN);
    return new UsCardResult(ndi);
}
}

```

The module registers itself with one line in `META-INF/services/com.capitalone.calc.spi.TenantCalculator`:

```
com.capitalone.calc.tenant.uscard.UsCardCalculator
```

A second tenant (`tenant-autofinance`)

Auto Finance starts from annual gross income, applies an effective tax rate, then subtracts obligations and an allowance per dependent. It has a different input, formula and settings, and shares no code with US Card.

```

record AutoFinanceInput(BigDecimal annualGrossIncome, BigDecimal
monthlyObligations, int dependents) {}
record AutoFinanceResult(BigDecimal ndi) {}

public final class AutoFinanceCalculator implements
TenantCalculator<AutoFinanceInput, AutoFinanceResult> {
    private static final TenantId ID = new TenantId("autofinance");
    private static final BigDecimal MONTHS_PER_YEAR = BigDecimal.valueOf(12);

    @Override public TenantId tenantId() { return ID; }
    @Override public String version() { return "autofinance-1"; }
    @Override public Class<AutoFinanceInput> inputType() { return
AutoFinanceInput.class; }

    @Override
    public AutoFinanceResult calculateNDI(TenantContext ctx, AutoFinanceInput
input) {
        if (input.annualGrossIncome().signum() <= 0) {
            throw new CalculateNDIRejectedException("annualGrossIncome must
be positive");
        }
        if (input.dependents() < 0) {
            throw new CalculateNDIRejectedException("dependents must not be
negative");
        }
        BigDecimal taxRate = ctx.settings().decimal("effective-tax-rate",

```

```

"0.25");
    BigDecimal perDependent = ctx.settings().decimal("per-dependent-
allowance", "300.00");

    BigDecimal monthlyGross =
input.annualGrossIncome().divide(MONTHS_PER_YEAR, 10,
RoundingMode.HALF_EVEN);
    BigDecimal monthlyNet =
monthlyGross.multiply(BigDecimal.ONE.subtract(taxRate));
    BigDecimal ndi = monthlyNet
        .subtract(input.monthlyObligations())

    .subtract(perDependent.multiply(BigDecimal.valueOf(input.dependents())));
    return new AutoFinanceResult(ndi.setScale(2,
RoundingMode.HALF_EVEN));
}
}

```

Shared dispatch (calc-app)

The registry is the only place shared code meets tenant code. It refuses to start if two modules claim the same tenant.

```

public final class TenantCalculatorRegistry {
    private final Map<TenantId, TenantCalculator<?, ?>> byTenant;

    /** Discovers every calculator on the runtime classpath. */
    @SuppressWarnings("rawtypes")
    public static TenantCalculatorRegistry discover() {
        Map<TenantId, TenantCalculator<?, ?>> found = new HashMap<>();
        for (TenantCalculator calculator :
ServiceLoader.load(TenantCalculator.class)) {
            register(found, calculator);
        }
        return new TenantCalculatorRegistry(found);
    }

    TenantCalculatorRegistry(Map<TenantId, TenantCalculator<?, ?>> byTenant)
{
    this.byTenant = Map.copyOf(byTenant);
}
}

```

```

    static void register(Map<TenantId, TenantCalculator<?, ?>> into,
TenantCalculator<?, ?> calculator) {
        if (into.putIfAbsent(calculator.tenantId(), calculator) != null) {
            throw new IllegalStateException(
                "More than one calculator registered for tenant " +
calculator.tenantId().value());
        }
    }

    public TenantCalculator<?, ?> forTenant(TenantId id) {
        TenantCalculator<?, ?> calculator = byTenant.get(id);
        if (calculator == null) {
            throw new TenantNotFoundException(id);
        }
        return calculator;
    }
}

```

The service is identical for every tenant: look up, convert, guard, call.

```

public final class CalculateNDIService {
    private final TenantCalculatorRegistry registry;
    private final TenantGuard guard;
    private final ObjectMapper mapper;

    public CalculateNDIService(TenantCalculatorRegistry registry, TenantGuard
guard, ObjectMapper mapper) {
        this.registry = registry;
        this.guard = guard;
        this.mapper = mapper;
    }

    public Computation calculateNDI(TenantContext ctx, JsonNode rawInput) {
        return run(registry.forTenant(ctx.tenantId()), ctx, rawInput);
    }

    private <I, O> Computation run(TenantCalculator<I, O> calculator,
TenantContext ctx, JsonNode rawInput) {
        I input = toInput(rawInput, calculator.inputType());
        O output = guard.run(ctx.tenantId(), () ->
calculator.calculateNDI(ctx, input));
        return new Computation(calculator.version(),
mapper.valueToTree(output));
    }
}

```

```

    }

    private <I> I toInput(JsonNode rawInput, Class<I> type) {
        try {
            return mapper.convertValue(rawInput, type);
        } catch (IllegalArgumentException e) {
            throw new InvalidInputException(e);    // 400 invalid-input
        }
    }
}

public record Computation(String calculatorVersion, JsonNode result) {}

/** Runs work inside one tenant's rate limit and concurrency limit. */
public interface TenantGuard {
    <T> T run(TenantId tenant, Supplier<T> work);
}

```

The platform's `ObjectMapper` rejects unknown, missing and null fields, and writes `BigDecimal` as a string. Tenant modules therefore need no JSON library, and sending US Card's body to Auto Finance's endpoint fails with 400.

Web layer (sketch)

The interceptor is the only code that reads `{tenantId}` from the URL. Everything after it receives a `TenantContext`.

```

@Component
class TenantContextInterceptor implements HandlerInterceptor {
    static final String ATTRIBUTE = TenantContext.class.getName();
    private final TenantSettingsProvider settings;

    TenantContextInterceptor(TenantSettingsProvider settings) {
        this.settings = settings;
    }

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse
response, Object handler) {
        @SuppressWarnings("unchecked")
        Map<String, String> pathVariables = (Map<String, String>)

```

```

request.setAttribute(HandlerMapping.URI_TEMPLATE_VARIABLES_ATTRIBUTE);
String pathTenant = pathVariables.get("tenantId");

var authentication = (JwtAuthenticationToken)
SecurityContextHolder.getContext().getAuthentication();
String tokenTenant =
authentication.getToken().getClaimAsString("tenant_id");

if (pathTenant == null || !pathTenant.equals(tokenTenant)) {
    throw new TenantMismatchException(); // 403, whether or not the
tenant exists
}
TenantId tenantId = new TenantId(pathTenant);
String requestId = Optional.ofNullable(request.getHeader("X-Request-
Id"))
    .orElseGet(() -> UUID.randomUUID().toString());

request.setAttribute(ATTRIBUTE, new TenantContext(tenantId,
requestId, settings.forTenant(tenantId)));
MDC.put("tenant", tenantId.value());
return true;
}

@Override
public void afterCompletion(HttpServletRequest request,
HttpServletRequest response,
                                Object handler, Exception ex) {
    MDC.remove("tenant");
}
}

```

A `HandlerMethodArgumentResolver` turns that request attribute into a controller parameter, so the controller has no `@PathVariable tenantId` to misuse.

```

@RestController
@RequestMapping("/v1/tenants/{tenantId}/calculate-ndi")
class CalculateNDIController {
    private final CalculateNDIService service;
    private final CalculateNDIRepository repository;

    CalculateNDIController(CalculateNDIService service,
CalculateNDIRepository repository) {
        this.service = service;
    }
}

```

```

        this.repository = repository;
    }

    @PostMapping
    CalculateNDIResponse calculateNDI(TenantContext ctx,
                                     @RequestHeader(name = "Idempotency-
Key", required = false) String key,
                                     @RequestBody CalculateNDIRequest body)
    {
        Computation computed = service.calculateNDI(ctx, body.input());
        return CalculateNDIResponse.of(repository.save(ctx, key,
body.input(), computed));
    }

    @GetMapping("/{calculationId}")
    CalculateNDIResponse get(TenantContext ctx, @PathVariable UUID
calculationId) {
        return repository.find(ctx, calculationId)
            .map(CalculateNDIResponse::of)
            .orElseThrow(NDINotFoundException::new);
    }
}

```

When `Idempotency-Key` is present, the controller first looks up the key for this tenant. A stored row with the same input is returned as is; a different input returns 409.

Data model

All tenants share one table, and Postgres row-level security filters every statement by tenant. A query that forgets its `WHERE tenant_id = ...` returns no rows, not another tenant's rows.

```

-- Run by the migration role (owns the table).
CREATE TABLE ndi_calculation (
    tenant_id          text          NOT NULL,
    calculation_id      uuid          NOT NULL,
    calculator_version  text          NOT NULL,
    idempotency_key     text,
    input               jsonb         NOT NULL,
    result              jsonb         NOT NULL,
    computed_at         timestampz    NOT NULL DEFAULT now(),
    PRIMARY KEY (tenant_id, calculation_id),

```

```

    UNIQUE (tenant_id, idempotency_key)
);

ALTER TABLE ndi_calculation ENABLE ROW LEVEL SECURITY;
ALTER TABLE ndi_calculation FORCE ROW LEVEL SECURITY;

-- A row is visible, and writable, only when it belongs to the tenant
-- the current transaction was opened for.
CREATE POLICY tenant_isolation ON ndi_calculation
    USING      (tenant_id = current_setting('app.tenant_id', true))
    WITH CHECK (tenant_id = current_setting('app.tenant_id', true));

-- The service connects as calc_app: not the owner, no BYPASSRLS.
GRANT SELECT, INSERT ON ndi_calculation TO calc_app;

```

The repository opens every transaction by naming the tenant. The third argument `true` makes the setting local to the transaction, so it cannot carry over on a pooled connection.

```

@Repository
class CalculateNDIRepository {
    private final JdbcClient jdbc;
    private final TransactionTemplate tx;

    CalculateNDIRepository(JdbcClient jdbc, TransactionTemplate tx) {
        this.jdbc = jdbc;
        this.tx = tx;
    }

    Optional<StoredNDI> find(TenantContext ctx, UUID calculationId) {
        return inTenant(ctx, () -> jdbc.sql("""
            SELECT tenant_id, calculation_id, calculator_version, result,
computed_at
                FROM ndi_calculation
                WHERE tenant_id = :tenant AND calculation_id = :id""")
            .param("tenant", ctx.tenantId().value())
            .param("id", calculationId)
            .query(StoredNDI.class)
            .optional());
    }

    private <T> T inTenant(TenantContext ctx, Supplier<T> work) {
        return tx.execute(status -> {
            jdbc.sql("SELECT set_config('app.tenant_id', :tenant, true)")

```

```

        .param("tenant", ctx.tenantId().value())
        .query(String.class)
        .single();
    return work.get();
});
}
}

```

Design points:

- **Two filters, on purpose.** The query names the tenant and the policy checks it again. Either one alone would be enough; together a bug in one is harmless.
- **No `TenantContext`, no query.** Every repository method takes a `TenantContext`, so data access without a tenant does not compile.
- **Idempotency keys are per tenant.** Two tenants can use the same key without colliding.
- **Results are append-only.** The service role has no `UPDATE` or `DELETE` grant.

The schema above was run on Postgres 16 as `calc_app`. Auto Finance could not read or count US Card's row, could not insert a row labelled `uscard`, and a session with no tenant set saw zero rows.

Runtime isolation

Every shared runtime resource is keyed by tenant id, so one tenant's load or failure stays with that tenant.

Concern	Mechanism	Effect on other tenants
Request rate	One rate limiter per tenant; over the limit returns 429	None: each tenant has its own quota
Concurrency	One semaphore bulkhead per tenant; full returns 503	A slow calculator cannot take every request thread
Calculator failure	Exceptions are caught at the service boundary and mapped to a problem response	A bug in one module fails only that tenant's endpoint
Configuration	Calculator receives only <code>tenants.<id>.*</code>	A tenant cannot read another tenant's settings

Concern	Mechanism	Effect on other tenants
Caching	Any cache key starts with the tenant id	No cross-tenant cache hits
Logs	<code>tenant</code> and <code>requestId</code> on every log line; <code>input</code> is not logged	Logs can be filtered and access-controlled per tenant
Metrics	<code>tenant</code> tag on request count, latency, 429s and 503s	Per-tenant dashboards and alerts

Limits come from platform defaults, overridden per tenant under `tenants.<id>.limits.*`.

The guard is a thin wrapper over Resilience4j, which creates one limiter and one bulkhead per name (sketch):

```
@Component
class Resilience4jTenantGuard implements TenantGuard {
    private final RateLimiterRegistry limiters;
    private final BulkheadRegistry bulkheads;

    Resilience4jTenantGuard(RateLimiterRegistry limiters, BulkheadRegistry
bulkheads) {
        this.limiters = limiters;
        this.bulkheads = bulkheads;
    }

    @Override
    public <T> T run(TenantId tenant, Supplier<T> work) {
        RateLimiter limiter = limiters.rateLimiter(tenant.value());
        Bulkhead bulkhead = bulkheads.bulkhead(tenant.value());
        return RateLimiter.decorateSupplier(limiter,
Bulkhead.decorateSupplier(bulkhead, work)).get();
    }
}
```

The guard runs only after the registry has found a calculator, so limiters are created for registered tenants only and their number is bounded.

Enforcing isolation

Each isolation rule has a check that fails the build or a test when the rule is broken, so the boundaries do not depend on code review.

Rule	Enforced by	Breaks at
A tenant module imports another tenant	Module classpath holds only <code>calc-spi</code>	Compile
Shared code names a tenant class	Tenant modules are runtime-scope dependencies of <code>calc-app</code>	Compile
A tenant module uses Spring, JDBC or an HTTP client	ArchUnit rule below	Unit test
Two modules claim one tenant	Registry throws at startup	Startup
A caller uses another tenant's path	API test: US Card token on Auto Finance path returns 403	Integration test
A query runs without a tenant	Row-level security returns no rows	Integration test
A tenant's formula changes by accident	Golden input and output cases inside the tenant module	Unit test

The ArchUnit rules live in `calc-app`'s tests, where every tenant module is on the classpath:

```

@AnalyzeClasses(packages = "com.capitalone.calc")
class TenantIsolationArchTest {

    @ArchTest
    static final ArchRule tenants_do_not_depend_on_each_other =
        slices().matching("com.capitalone.calc.tenant.
(*)..").should().notDependOnEachOther();

    @ArchTest
    static final ArchRule tenants_depend_only_on_the_contract =
        classes().that().resideInAPackage("com.capitalone.calc.tenant..")
            .should().onlyDependOnClassesThat()
                .resideInAnyPackage("com.capitalone.calc.spi..",
"com.capitalone.calc.tenant..", "java..");

    @ArchTest
    static final ArchRule shared_code_does_not_name_tenants =

noClasses().that().resideOutsideOfPackage("com.capitalone.calc.tenant..")

.should().dependOnClassesThat().resideInAPackage("com.capitalone.calc.tenant.
.");
}

```

What was checked for this design

- **Compiled and run:** the contract, both calculators, the registry and the service, with 16 checks passing. These include both worked examples (1850.00 and 2550.00), each tenant's body rejected on the other's endpoint, and settings filtered per tenant.
- **Compile boundary:** an Auto Finance class importing `UsCardCalculator`, and a shared class naming it, both failed to compile.
- **Row-level security:** the schema and policy, run on Postgres 16.
- **Not run:** the Spring web layer, the Resilience4j guard and the ArchUnit rules. Their libraries could not be downloaded in the environment this was drafted in.

Onboarding a new tenant

Adding a tenant adds one module and one dependency line. No shared class, route or table changes.

1. Create `tenants/tenant-<id>` with `calc-spi` as its only dependency.
2. Define the tenant's input and output records and implement `TenantCalculator`, returning the new slug from `tenantId()`.
3. Add the calculator's class name to `META-INF/services/com.capitalone.calc.spi.TenantCalculator`.
4. Add golden input and output cases as unit tests in the module.
5. Add the module to `calc-app` as a runtime-scope dependency. The ArchUnit rules cover it by package pattern.
6. Add `tenants.<id>.*` settings and limits to configuration.
7. Issue credentials whose token carries `tenant_id = <id>`.
8. Deploy. `POST /v1/tenants/<id>/calculate-ndi` is live.

Removing a tenant is the reverse of steps 5 and 7. Its endpoint then rejects every caller, and its stored rows stay unreadable until they are archived or deleted.

Open decisions

These are the assumptions most likely to change the design.

- ☐ **Stack.** Java and Spring Boot are assumed. Which language and framework will this be built in?
- ☐ **Tenant in the URL.** The design uses a path segment. A subdomain per tenant (`uscard.api.capitalone.com`) changes only the interceptor.
- ☐ **How tenants are added.** The design assumes a deploy per new tenant. If logic must change without a deploy, calculators become plugin jars or rule definitions, and the registry loads them at runtime.
- ☐ **Storing results.** If the API can be stateless, drop the `GET` endpoint, the table and idempotency.
- ☐ **Depth of data isolation.** One table with row-level security is assumed. Does any tenant need its own schema or database?
- ☐ **Authentication.** A JWT with a `tenant_id` claim is assumed. Are API keys or mutual TLS required instead?
- ☐ **Long calculations.** If any tenant's calculation can run for seconds, add a per-tenant timeout or an asynchronous job API.